

Real Estate Operations Analytics – Technical Documentation

*** Project:**

New Cairo Real Estate Operations Analytics (Data Warehouse & BI Solution)

*** Author:**

Aisha Abubhashima

*** Stack:**

Python (Google Colab) → Microsoft SQL Server → Power BI Desktop

*** Domain:**

Real estate resale brokerage (Sale + Rental operations), New Cairo, Egypt

1. Business Case

A rapidly growing real estate resale company (operating both sale and rental transactions on third-party inventory – not a property developer) was running its operations on manual, paper-based tracking. Decision-makers had no centralized system to monitor sales performance, marketing effectiveness, employee productivity, or operational expenses.

*** Objective:**

Design and build an end-to-end data analytics solution – synthetic data generation, a normalized relational data warehouse, and an interactive BI dashboard – to support data-driven operational decisions and to serve as a reproducible, interview-defensible capstone project.

*** Pipeline:**

Python (synthetic data generation & business-rule enforcement)

↓

Google Sheets (live export, human-readable staging layer)

↓

CSV export

↓

SQL Server (relational data warehouse, star-schema constraints)

↓

Power BI Desktop (semantic model, DAX measures, 3-page dashboard)

2. Data Generation Layer (Python / Google Colab)

2.1 Tooling and Library Choices

- Library | Purpose | Rationale

- **pandas** | Tabular data construction (DataFrame)
| Native fit for row-wise record generation and direct export to CSV/Sheets.
- **random** | Weighted/unweighted categorical sampling, seeded reproducibility
| `random.seed(N)` guarantees the same synthetic dataset is regenerated identically on every run – critical for a reproducible thesis project and for keeping SQL/Power BI in sync with a specific data snapshot.
- **numpy (np.random)** | Vectorized/weighted sampling for high-volume tables (Leads, Sales, Rentals)
| Used specifically where `random.choices` with probability weights needed to scale efficiently across 1,000+ rows (e.g., lead-to-campaign assignment).
- **datetime, timedelta** | Date/time arithmetic
| Needed to move beyond year-level granularity to full datetime (day + hour + minute) precision for operational cycle-time analysis.
- **gsread + google.colab.auth + googleapiclient.discovery** | Live read/write to Google Sheets
| Chosen as a staging/verification layer so the data owner could visually inspect generated tables before committing them to SQL, without leaving the notebook environment.

A global random seed is set once per generation block (a distinct seed per table: 42 for Properties, 101 for Clients, 202 for Sellers, etc.) so that each table's randomness is independent and reproducible, while the overall dataset composition remains stable across re-runs.

2.2 Design Pattern: Shared Work Calendar

Because the operational KPIs required realistic entry timestamps (not just year-level dates), a shared calendar utility was built once and reused across every table that needed a "when was this entered / who entered it" dimension:

- **WEEKEND_DAYS = {4, 5}** – Friday & Saturday, matching the Egyptian work week.
- **HOLIDAYS – a set()** – of Egyptian national holidays (fixed dates) and approximate Islamic holidays (Eid al-Fitr, Eid al-Adha, Islamic New Year, Prophet's Birthday) for 2023–2026, expanded via a **expand_range()** helper for multi-day holidays.
- **is_working_day(date)** – rejects weekends and holidays.

- **WORK_START_HOUR = 9 , WORK_END_HOUR = 17** – bounds random entry times to realistic office hours.

- **DATA_ENTRY_STAFF_WINDOWS** – a dictionary mapping each Data Management employee to their actual employment window (hire_date, exit_date or None), so an employee can never be credited with data entry outside the period they were actually employed.

- **generate_work_datetime(year)** – the core reusable function: retries up to 200 times to find a valid working day/hour combination, then attributes the entry to a currently-employed staff member via pick_available_employee().

*** Rationale ***

This directly answers the original business question – "what is our realistic data-entry throughput, accounting for weekly/annual holidays and staff turnover?" – at the source, rather than trying to approximate it later in DAX.

2.3 Design Pattern: Business-Rule Functions

Rather than generating each field independently at random, domain constraints were encoded as pure functions called during row construction, so invalid combinations are structurally impossible:

- **apply_property_domain_rules(finishing, usage)** – enforces that Core & Shell / Semi Finished units can only be Unfurnished and Sale-only (no rental of an unfinished shell); Fully Finished units may be furnished and rented.

- **get_seller_business_logic(seller_type_code)** – enforces that Developers (DV) only ever list units for Sale, Hospitality companies (HS) only ever offer Rent, while individual Owners (OW) and Co-Brokers (BR) can do either (with a small 4% probability of "Both").

- **generate_client_budget_and_preference(client_type)** – links client type (Buyer/Seller/Tenant/Landlord) to a logically consistent deal type and budget band: Buyers/Sellers get multi-million EGP sale budgets, Tenants/Landlords get monthly rental budgets – preventing nonsensical outputs like a "Tenant" with an 8,000,000 EGP budget.

- **classify_client(min_budget, usage, deal_type)** – derives a Class A/B/C segmentation from budget thresholds that differ by usage type and deal type (residential vs. commercial, sale vs. rent).

2.4 Table-by-Table Generation Summary

- Table | Row Count | Key Generation Logic

- **Dim_Properties** | 500
| PropertyID encodes region + category + usage + type + sequence (e.g., NC_1aA0000001). Location (Square/District/Zone) is drawn from a year-indexed GEOGRAPHIC_MAP so location and vintage stay logically linked. Financials (Amount_Downpayment, RemainingAmount) differ by SellingOption.

CreatedDateTime/EnteredByEmployeeID sourced from the shared work-calendar utility. RentedDateTime populated only for units where SellingOption='Rent' and UnitStatus='Rented', generated with a seasonal bias (75% probability within the 15-May-15-Sep rental peak season, 25% off-season) to reflect real market seasonality.

- **Dim_Clients** | 500

| Demand-side records (not yet linked to a transaction). Budget/deal type derived via the business-rule function above| .

- **Dim_Sellers** | 200

| Supply-side partners. A post-generation resync_step (resync_property_sellers()) re-assigns every property's SellerID to a seller who is actually eligible to offer that deal type – added after discovering the initial independent-random assignment produced referential mismatches between Dim_Properties and Dim_Sellers.

- **Dim_Departments_Roles** | 20

| Static lookup, hand-authored (not randomized) to represent a realistic 8-department organizational structure.

- **Dim_Employees** + • **Dim_Employee_Career_History** | 26 / 38

| Hand-authored (not randomized) to guarantee a coherent, auditable career history per employee (hire → promotion → resignation/termination), including a ManagerID self-referencing hierarchy. Dim_Employee_Career_History is modeled as an event_log (one row per HR event: Hire/Promotion/Resignation/Termination) rather than an "old value/new value" SCD table, chosen for simplicity of downstream DAX (EventType -based filtering) over a classic Type-2 SCD.

- **Dim_Marketing_Campaigns** | 10

| Static campaign metadata (name, platform, budget, launch year) – separated into its own dimension after discovering the original design repeated the campaign budget on every lead row, which would double-count budget under a SUM() aggregation.

- **Dim_Marketing_Leads** | 1,200

| ClientID linked directly to Dim_Clients (allowing the same client to generate multiple leads over time, enabling New vs. Returning client analysis). ReceivedDateTime is not bounded by the work calendar (leads arrive via digital ads 24/7, unlike internal data-entry).

- **Fact_Leads_Operations** | 1,200

| One row per lead-handling event. OperationDateTime is derived by taking the lead's ReceivedDateTime, adding a random response delay, then snapping_forward to the next valid working slot (snap_to_next_working_slot()) – ensuring an operation can never be logged before its lead arrived, and never outside working hours.

- **Fact_Sales_Transactions** | ~360-400

| Only Qualified / Assigned leads and Sale-type properties are eligible. UnitPrice_EGP = Amount_Downpayment + RemainingAmount. ClientID is pulled through from the parent lead (lead_to_client map) and an IsFirstPurchase flag is computed via groupby('ClientID')['SaleDate'].transform('min') –

pushing the "new vs. returning buyer" logic into Python so it survives database rebuilds (avoiding reliance on a Power BI calculated column).

- **Fact_Rental_Transactions** | ~35-40

| Added in a later iteration once it was recognized that ~50% of a resale brokerage's real activity is rental, not sale, and the original schema had no rental fact table. Company commission modeled as two months' rent (one month from landlord, one from tenant) per local market convention. RentalDate generation uses the same 75/25 seasonal-bias logic as RentedDateTime above.

- **Fact_Monthly_Expenses** | ~120

| Synthetic operating-expense ledger (Office Rent, Electricity, Office Supplies) added to make `Net Profit` a genuine, defensible figure rather than raw revenue. Electricity and Supplies costs are derived from the active headcount per month (via HireDate/ExitDate), not independent randoms – so expenses scale believably as the company grows.

2.5 Validation Pattern

Every generation cell is paired with a lightweight audit cell, **e.g.:**

```
first_purchase_check =  
df_fact_sales_transactions.groupby('ClientID')['IsFirstPurchase'].sum()  
invalid_clients = first_purchase_check[first_purchase_check != 1]  
print(f"Audit: clients with incorrect IsFirstPurchase count =  
{len(invalid_clients)}")
```

This "**generate → assert referential/logical integrity → export**" pattern is repeated for every table (duplicate-key checks, orphan foreign-key checks, budget min≤max checks, etc.) so schema-breaking bugs are caught in the notebook, not in SQL Server.

2.6 Export

Each table is exported twice:

- **Google Sheets (gsread)** – a live, shared spreadsheet used as a human-readable checkpoint (folder → spreadsheet → per-table worksheet tab, cleared and rewritten on each run).

- **CSV** – one file per table, written to a local Colab folder and zipped (shutil.make_archive) for download, then imported into SQL Server via SSMS's Import/Export Flat File.

3. Data Warehouse Layer (Microsoft SQL Server)

3.1 Schema Design

A *star schema* with 8 dimension/lookup tables and 4 fact tables:

- **Dimensions:** Dim_Properties, Dim_Clients, Dim_Sellers, Dim_Departments_Roles, Dim_Employees, Dim_Employee_Career_History, Dim_Marketing_Campaigns, Dim_Marketing_Leads
- **Facts:** Fact_Leads_Operations, Fact_Sales_Transactions, Fact_Rental_Transactions, Fact_Monthly_Expenses
- **Bridge:** Dim_Marketing_Leads sits at the center of the model, acting as the bridge between marketing acquisition (Dim_Marketing_Campaigns, Dim_Clients) and the operational/transactional facts.

3.2 Key Constraints and Referential Integrity

- Every table uses a natural, business-meaningful **VARCHAR** primary key (e.g., PropertyID, EmployeeID) rather than a surrogate integer, to keep the keys self-descriptive and traceable back to the Python-generated encoding scheme.
CHECK constraints enforce enumerated domains at the database layer (e.g., Status IN ('Active', 'Resigned', 'Terminated', 'Promoted'), PaymentMethod IN ('Installments', 'Cash / Lump Sum')) – a deliberate defense-in-depth choice so bad data cannot enter even if the Python validation step were skipped.
- Foreign keys enforce the full dependency graph, including a **self-referencing FK** on Dim_Employees.ManagerID → Dim_Employees.EmployeeID for the reporting hierarchy.
- Dim_Properties.EnteredByEmployeeID and Dim_Sellers.EnteredByEmployeeID are **FK**'d to Dim_Employees, enabling "who entered this record" auditing directly in SQL.

3.3 DROP/CREATE Ordering

- **DROP TABLE IF EXISTS** statements are ordered *fact tables and dependent tables first, root dimensions last* (e.g., Fact_Monthly_Expenses → Fact_Rental_Transactions → Fact_Sales_Transactions → Fact_Leads_Operations → Dim_Employee_Career_History → Dim_Properties → Dim_Sellers → Dim_Employees → ...) – the reverse of creation order – so that a full database rebuild never fails on a dangling foreign key. This ordering was corrected after an early version of the script attempted to drop parent dimension tables while child fact tables still referenced them.

3.4 Notable Schema Evolution

- Addition | Reason

- **CreatedDateTime, EnteredByEmployeeID, RentedDateTime** on Dim_Properties
| Required to answer the original data-entry-throughput business question at day/hour granularity.

- **Dim_Marketing_Campaigns** (new table)
| Removed a repeated-value trap where CampaignBudget_EGP was stored once per lead row, which would inflate `SUM()`-based CPL/CPA/ROAS calculations.

- **ClientID, IsFirstPurchase** on Fact_Sales_Transactions
| Enables New vs. Returning Client analysis; deliberately generated in Python (not as a Power BI calculated column) so the flag survives repeated database rebuilds.

- **Fact_Rental_Transactions** (new table)
| The company's core business model includes rental brokerage, not just sales — omitting it would misrepresent ~50%+ of actual revenue-generating activity.

- **Fact_Monthly_Expenses** (new table)
| Required to compute a genuine `Net Profit` KPI rather than presenting gross revenue as if it were profit.

4. Analytics Layer (Power BI)

4.1 Semantic Model / Relationships

All relationships are **Many-to-One, Single** cross-filter direction, unless noted:

- Dim_Properties[SellerID] → Dim_Sellers[SellerID] (**Active**)
- Dim_Employees[CurrentRoleID] → Dim_Departments_Roles[RoleID] (**Active**)
- Dim_Employee_Career_History[EmployeeID] → Dim_Employees[EmployeeID] (**Active**)
- Dim_Marketing_Leads[ClientID] → Dim_Clients[ClientID] (**Active**)
- Dim_Marketing_Leads[CampaignID] → Dim_Marketing_Campaigns[CampaignID] (**Active**)
- Fact_Leads_Operations[LeadID] → Dim_Marketing_Leads[LeadID] (**Active**)
- Fact_Leads_Operations[ModeratorID] → Dim_Employees[EmployeeID] (**Active**)
- Fact_Leads_Operations[SalesAdminID] → Dim_Employees[EmployeeID] (**Inactive** – dual role reference to the same dimension resolved via USERRELATIONSHIP() where needed)
- Fact_Sales_Transactions[LeadID/PropertyID/SalesRepID] → ... (**Active**); [TeamLeaderID] → Dim_Employees (**Inactive**, same reason)
- Fact_Rental_Transactions[LeadID/PropertyID/SalesRepID] → ... (**Active**); [TeamLeaderID] (**Inactive**)
- Fact_Sales_Transactions[SaleDate] → Dim_Date[Date] and • Fact_Monthly_Expenses[ExpenseDate] → Dim_Date[Date] (**Active** – Time Intelligence)

Design decision

Dim_Properties, Dim_Sellers, and Fact_Leads_Operations were deliberately not given a direct relationship to Dim_Date, because a direct link would create a second filter path to Dim_Date alongside the existing indirect path (via Fact_Sales_Transactions), producing a circular/ambiguous relationship graph that Power BI rejects at refresh time. Where day/week/month breakdowns were still needed for these tables (e.g., an entry-activity heatmap), the weekday/month values are derived locally via calculated columns (FORMAT()/WEEKDAY()) directly from the table's own datetime column, avoiding the relationship entirely.

4.2 Key DAX Measures (by purpose)

• Category | Measure | Logic Summary

• **Revenue** | Total Revenue
| [Total Company Commission] – i.e., the firm's actual commission income (sale + rental), not gross transacted property value, which would overstate revenue by two orders of magnitude for a brokerage.

• **Revenue** | Total Company Commission / Total Employees Commission
| Sum of CompanyCommission_EGP / AgentCommission_EGP across both Fact_Sales_Transactions and Fact_Rental_Transactions.

• **Profitability** | Total Expenses
| SUM(Fact_Monthly_Expenses[Amount_EGP]), filtered to the model's date range.

- **Profitability** | Net Profit
| $[Total\ Revenue] - [Total\ Expenses]$.
- **Marketing efficiency** | CPL, CPA, ROAS
| Cost-per-lead, cost-per-acquisition, and return-on-ad-spend, computed from `Dim_Marketing_Campaigns[Budget_EGP]` against $[Total\ Leads] / [Total\ Deals]$ $/ [Total\ Revenue]$ respectively.
- **Funnel** | Total Leads, Qualified Leads, Total Deals, Conversion Rate
| Funnel-stage counts from `Fact_Leads_Operations[OperationStatus]` and deal counts from the two transaction fact tables.
- **Client behavior** | New Clients, Returning Clients, Total Buyers
| Built on `Fact_Sales_Transactions[IsFirstPurchase]` (sale-only by design decision – rental "returning tenant" behavior was scoped out to keep the KPI defensible and unambiguous for the presentation).
- **Cycle time** | Avg Deal Life Cycle (Days), Avg Rental Deal Cycle (Days), Avg Unit Lifecycle (Days)
| `DATEDIFF()` between lead receipt and deal closure (sale/rental, tracked as separate measures deliberately, to allow comparing the two conversion speeds side-by-side rather than averaging them into one misleading number), and between unit entry and sale/rental date respectively.
- **Workforce** | Current Headcount, Current Payroll, Turnover Rate, Hiring Count, Promotion Count, Employee Movement Value
| Derived from `Dim_Employees/Dim_Employee_Career_History`, filtered on `ISBLANK(ExitDate)` for "currently active" logic and on `EventType` for HR-event counts.

4.3 Dashboard Structure (3 pages)

- **Page 1 – Executive Overview**
KPIs: Total Revenue, Expenses, Net Profit, Sale Revenue, Rental Revenue, CPL, CPA.
Charts: Expense Category breakdown (pie), Revenue by Year-Quarter (line), Company Commission Sale-vs-Rent split (donut), Revenue by Property Type (column), CPL/ROAS scatter by campaign budget, Revenue by District (bar), Rent/Sale slicer (via `Dim_Properties[DealCategory]`).
Purpose: single-glance business health check for leadership; explicitly excludes gross transacted property value from the headline Revenue figure to avoid misleading stakeholders.
- **Page 2 – Operations Analysis**
KPIs: Total Deals, Total Sellers, Total Buyers, Conversion Rate %, Avg Unit Lifecycle (Days), Avg Entries/Day.
Charts: Deals by Year-Quarter (line), Total Leads → Qualified Leads → Total Deals (funnel), Avg Sale Deal Cycle (gauge, target = 21 days), Avg Rental Deal Cycle (gauge, target = 90 days – one seasonal rental cycle), Lead Operations Outcome (donut), Marketing Leads by Platform (bar), Rent/Sale slicer (via `Dim_Properties[DealCategory]`).
Purpose: operational efficiency – where deals are won/lost in the funnel, and how quickly.

- **Page 3 – Employees Performance**

KPIs: Departments, Employees, Males, Females, Current Payroll, Hiring Count, Promotion Count.

Charts: Deals-Employees Performance (bar, revenue per sales rep, color-highlighted top performers), Males & Females by Department (stacked column), Salary vs. Tenure by Department (scatter – used to visually test/refute the assumption that retention is purely salary-driven), Hiring & Promotion by Year (line), Employee Movement Value (waterfall: Hire/Promotion/Resignation/Termination), Turnover Rate (gauge) , Rent/Sale slicer (via Dim_Properties[DealCategory]).

Purpose: operational (not HR-administrative) view of workforce productivity and stability, explicitly framed for an Operations Analyst audience.

4.4 Notable Debugging Lessons (worth documenting for future reference)

Calculated columns are not persisted through a data-source rebuild. Any Power BI *calculated column* (as opposed to a column native to the SQL source) is recomputed from the model, not stored server-side – repeated database rebuilds during iterative development silently wiped columns like **IsFirstPurchase**, **SaleDateIfSold**, **UnitCycleDays**, **DealCycleDays**, and **DealCategory** multiple times. The long-term fix was to migrate frequently-needed derived columns (e.g., `IsFirstPurchase`) back into the Python/SQL layer wherever feasible.

Ambiguous/cyclic relationship paths are rejected by the Vertipaq engine at refresh time ("A cyclic reference was encountered during evaluation") whenever two relationship paths connect the same pair of tables – resolved by explicitly deciding, per table, one deliberate filter path to **Dim_Date** rather than adding a "just in case" second relationship.

5. Summary

The final solution comprises 12 SQL Server tables (8 dimensions, 4 facts) sourced from a fully reproducible Python/pandas synthetic-data pipeline, surfaced through a 3-page Power BI dashboard built on ~35 DAX measures. Key design decisions – separating gross transacted value from actual commission revenue, modeling sale and rental as parallel but distinct fact tables, and deriving data-entry throughput from a realistic Egyptian work calendar – were made specifically to keep every KPI on the dashboard defensible against domain-expert scrutiny (e.g., in a technical interview or academic defense), rather than optimizing for visual completeness alone.